



COMP30026

Models of Computation

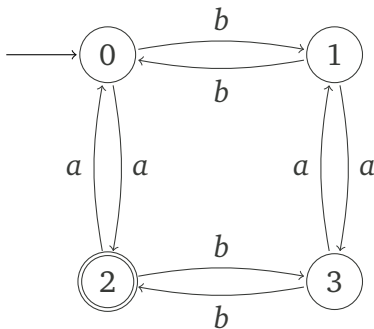
Cezary Kaliszyk and William Umboh

Lecture 12

NFA Determinisation and DFA Minimisation

Reminder DFAs

Which language is recognised by this machine?





Regular Languages

Definition

A language is **regular** iff there is a finite automaton that recognises it.

Soon we will see that not all languages are regular

Regular Operations

Let A and B be languages (i.e. sets of strings).

The **regular operations** are:

- **Union:** $A \cup B$
- **Concatenation:** $A \circ B = \{xy \mid x \in A, y \in B\}$
- **Kleene star:** $A^* = \{x_1x_2 \cdots x_k \mid k \geq 0, \text{ each } x_i \in A\}$

Note that the empty string, ϵ , is always in A^* .

Regular Operations: Example

Let $A = \{aa, abba\}$ and $B = \{a, ba, bba, bbba\}$. Then,

$$A \cup B = \{a, aa, abba, ba, bba, bbba\},$$

$$A \circ B = \{aaa, abbaa, aaba, abbaba, aabba, abbabba, \dots\},$$

$$A^* = \left\{ \begin{array}{l} \epsilon, aa, abba, aaaa, aaabba, abbaaa, abbaabba, \\ aaaaaa, aaaaabba, aaabbbaa, aaabbaabba, \dots \end{array} \right\}.$$



Closure

Theorem

If A and B are regular languages, then $A \cup B$, $A \circ B$, and A^ are also regular.*

That is, the regular languages are **closed** under regular operations.

How to prove? **Nondeterminism**.



Nondeterminism

So far: **deterministic** finite automata (DFA)

Multiple possible transitions for same symbol:

NFAs (non-deterministic finite automata).



Nondeterminism

So far: **deterministic** finite automata (DFA)

Multiple possible transitions for same symbol:

NFAs (non-deterministic finite automata).

$$\left\{ w \mid \begin{array}{l} w \in \{0, 1\}^* \text{ has length 3 or more,} \\ \text{and the third last symbol in } w \text{ is 1} \end{array} \right\}$$

Lets think of an NFA and a DFA for above language



Nondeterminism

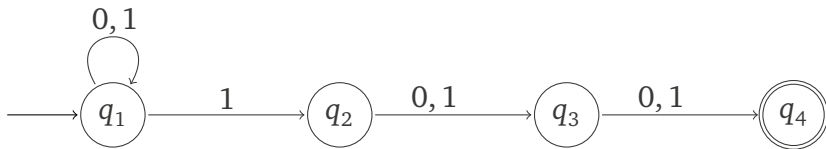
So far: **deterministic** finite automata (DFA)

Multiple possible transitions for same symbol:

NFAs (non-deterministic finite automata).

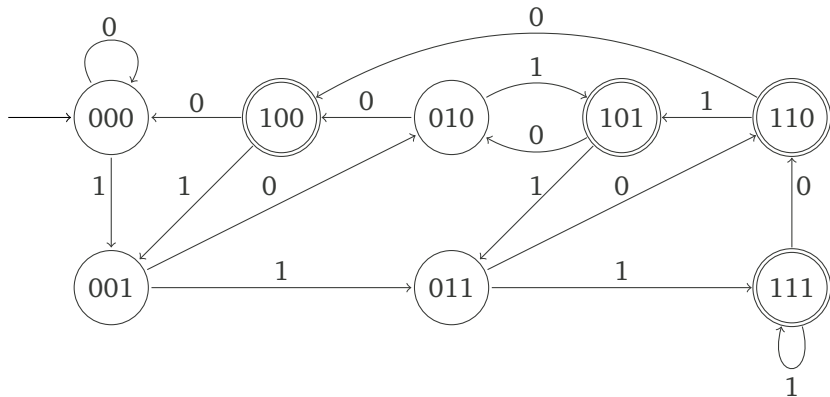
$$\left\{ w \mid \begin{array}{l} w \in \{0, 1\}^* \text{ has length 3 or more,} \\ \text{and the third last symbol in } w \text{ is } 1 \end{array} \right\}$$

Lets think of an NFA and a DFA for above language

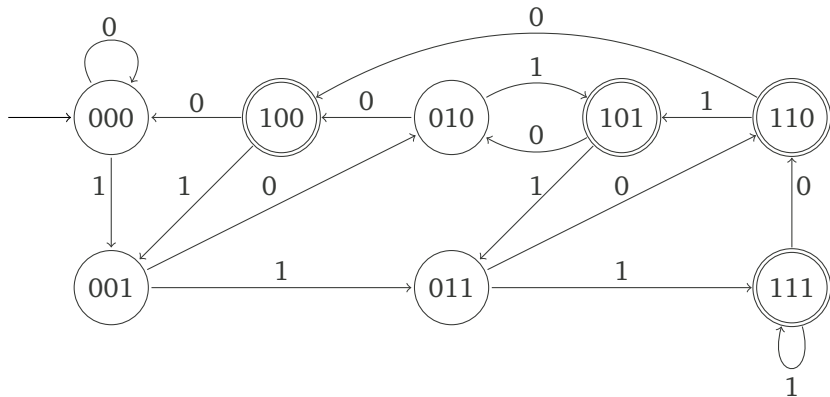


Note: **No** transitions from q_4 , and **two** possible transitions when we meet a 1 in state q_1 .

Nondeterminism

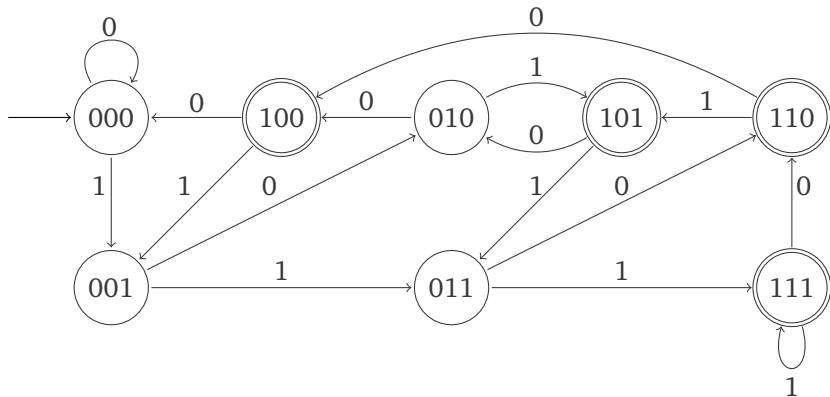


Nondeterminism



This is the simplest DFA that will do the job!

Nondeterminism



This is the simplest DFA that will do the job!

The NFA for same language a lot simpler...



Epsilon Transitions

NFAs may move states without consuming input.

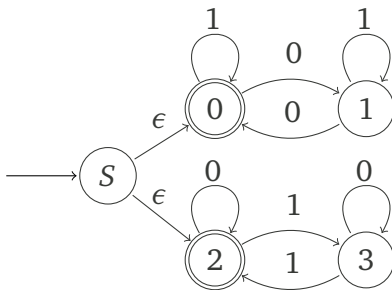
Such a transition is an ϵ transition.

Epsilon Transitions

NFAs may move states without consuming input.

Such a transition is an ϵ transition.

Among other things, this gives us an easy way to construct a machine to recognise the **union** of two languages:





Formal Definition of NFA

For any alphabet Σ let Σ_ϵ denote $\Sigma \cup \{\epsilon\}$.

An **NFA** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of **states**,
- Σ is a finite **alphabet**,
- $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ are the **accept states**.

Reminder: $\mathcal{P}(Q)$ are all subsets of Q



NFA Acceptance, Formally

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA and
let $w = v_1 v_2 \cdots v_n$ where each v_i is a member of Σ_ϵ .

N **accepts** w iff there exists a sequence of states $r_0, r_1, \dots, r_n$,
with each $r_i \in Q$, such that

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, v_{i+1})$ for $i = 0, \dots, n-1$
3. $r_n \in F$



DFAs vs NFAs

Definition

Machines are **equivalent** if and only if they recognize the same language.



DFAs vs NFAs

Definition

Machines are **equivalent** if and only if they recognize the same language.

Theorem

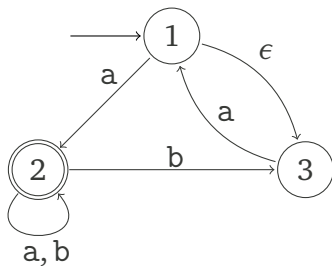
Every NFA has an equivalent DFA.

How? **Simulate** an NFA using a DFA.



DFAs vs NFAs

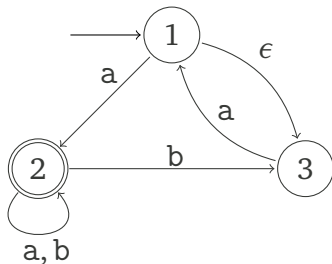
Consider the NFA on the right.
We construct an equivalent DFA.



DFAs vs NFAs

Consider the NFA on the right.
We construct an equivalent DFA.

The DFA's start state is $\{1, 3\}$.
From $\{1, 3\}$ a takes us to $\{1, 2, 3\}$.
From $\{1, 2, 3\}$, a takes us back to $\{1, 2, 3\}$, b takes us to $\{2, 3\}$.



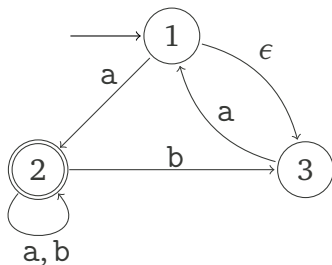
	a	b
$A=\{1,3\}$	B^+	-
$B^+=\{1,2,3\}$	B^+	C^+
$C^+=\{2,3\}$	B^+	C^+

DFAs vs NFAs

Consider the NFA on the right.
We construct an equivalent DFA.

The DFA's start state is $\{1, 3\}$.
From $\{1, 3\}$ a takes us to $\{1, 2, 3\}$.
From $\{1, 2, 3\}$, a takes us back to $\{1, 2, 3\}$, b takes us to $\{2, 3\}$.

Any state S which **contains** an accept state from the NFA will be an accept state for the DFA (here marked $^+$).

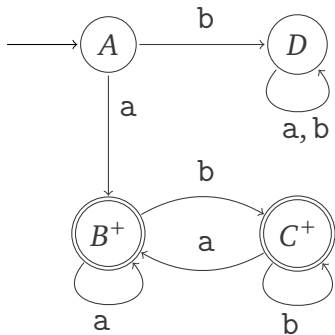


	a	b
$A=\{1,3\}$	B^+	-
$B^+=\{1,2,3\}$	B^+	C^+
$C^+=\{2,3\}$	B^+	C^+

DFAs vs NFAs

Here is the equivalent DFA that we derived.

We add (dead) state D that corresponds to the empty set.



	a	b
$A = \{1, 3\}$	B^+	D
$B^+ = \{1, 2, 3\}$	B^+	C^+
$C^+ = \{2, 3\}$	B^+	C^+
$D = \emptyset$	D	D